Architecture Document

1. Application:

The TB and Lung Disease Diagnostic System is built on a modern, distributed architecture. Medical imaging applications require a delicate balance between **data integrity**, **low-latency response**, and **high computational throughput**. To achieve this, the system employs a hybrid architectural strategy.

1.1 Microservices Architecture

The core philosophy of our microservices design is **Domain-Driven Design (DDD)**. By breaking the application into smaller, autonomous services, we ensure that the failure of one component (e.g., the reporting service) does not prevent the core diagnostic engine from functioning.

1.1.1 API Gateway & Orchestration

The API Gateway acts as the single entry point for all client applications (Web, Mobile, or Hospital Management Systems). It handles:

- **Request Routing:** Directing traffic to the appropriate microservice.
- **Authentication/Authorization:** Validating JWT tokens against the user database before allowing image uploads.
- **Rate Limiting:** Ensuring the Inference Service is not overwhelmed by too many simultaneous requests from a single source.

1.1.2 Deep Learning Inference Service (Core Service)

This is the most critical microservice. It encapsulates the hybrid model consisting of the **EfficientNetB3** and **DenseNet121** backbones.

- **Logic:** It receives pre-processed image tensors. It runs these tensors through the dual-backbone architecture. The service specifically manages the **Concatenation Layer**, where the global scalable features from EfficientNet and the dense gradient features from DenseNet are merged.

- **Resource Management:** This service is hosted on GPU-optimized instances (e.g., NVIDIA T4) to ensure that the 4-class classification (COVID, TB, Pneumonia, Normal) happens in sub-second intervals.

1.1.3 Image Management & Storage Service

This service manages the lifecycle of the 13,209+ images used and generated by the system.

- **Persistence:** It interacts with a specialized Blob Storage (like S3 or GCS) to store high-resolution X-rays.

- **Metadata Tracking:** It records clinical metadata such as patient IDs, timestamps, and image origin, ensuring that every diagnostic result is traceable back to a specific file.

1.1.4 Diagnostic Reporting Service

Once the Inference Service produces a Softmax output (probabilities for the 4 classes), this service translates those numerical values into a clinical report.

- **Template Logic:** It maps the highest index (0-3) to the corresponding disease label.

- **Data Aggregation:** It combines the confidence score with the original image and any relevant metadata to generate a PDF or FHIR-compliant medical record.

1.2 Event-Driven Architecture (EDA)

Medical diagnostics often involve "bursty" workloads where a hospital might upload a batch of 50 X-rays at once. An Event-Driven approach ensures the system remains resilient under this pressure.

1.2.1 Asynchronous Processing via Message Broker

The system uses a Message Broker (such as RabbitMQ or Apache Kafka) to manage the flow of data between services. Instead of a service waiting for a response (synchronous), it emits an event and moves on.

- **`IMAGE_UPLOADED` Event:** Triggered when the Image Service saves a file. This event contains the URI of the image.

- **`PREPROCESSING_READY` Event:** Triggered after the image has been resized to 224x224 and normalized. This event signals the Inference Service to begin the deep learning pass.

- **`DIAGNOSIS_COMPLETED` Event:** Once the Softmax probabilities are calculated, this event is published to notify the Reporting Service and the User UI.

1.2.2 Decoupling and Scalability

EDA allows the **Inference Service** to scale independently. If the message queue for `PREPROCESSING_READY` grows too long, the system can automatically spin up additional GPU workers to clear the backlog, ensuring that clinicians never face "system busy" errors during peak hours.

1.3 Serverless Architecture

To optimize operational costs and handle unpredictable traffic, we offload "stateless" and "short-lived" tasks to a serverless environment (Function-as-a-Service).

1.3.1 Serverless Image Preprocessing

Preprocessing 13,000+ images for training or individual images for inference involves repetitive math (division by 255, bilinear interpolation for resizing).

- **Execution:** We use serverless functions (like AWS Lambda or Google Cloud Functions) to handle these tasks. When an image hits the storage bucket, the function triggers, transforms the image to a **224x224x3 tensor**, and saves it to a "Ready" folder.

- **Benefit:** We only pay for the milliseconds the function runs, rather than maintaining a 24/7 server for simple image math.

1.3.2 Real-time Monitoring & Logging Functions

The system generates a vast amount of logs regarding model performance (Loss, Accuracy, and per-class confidence).

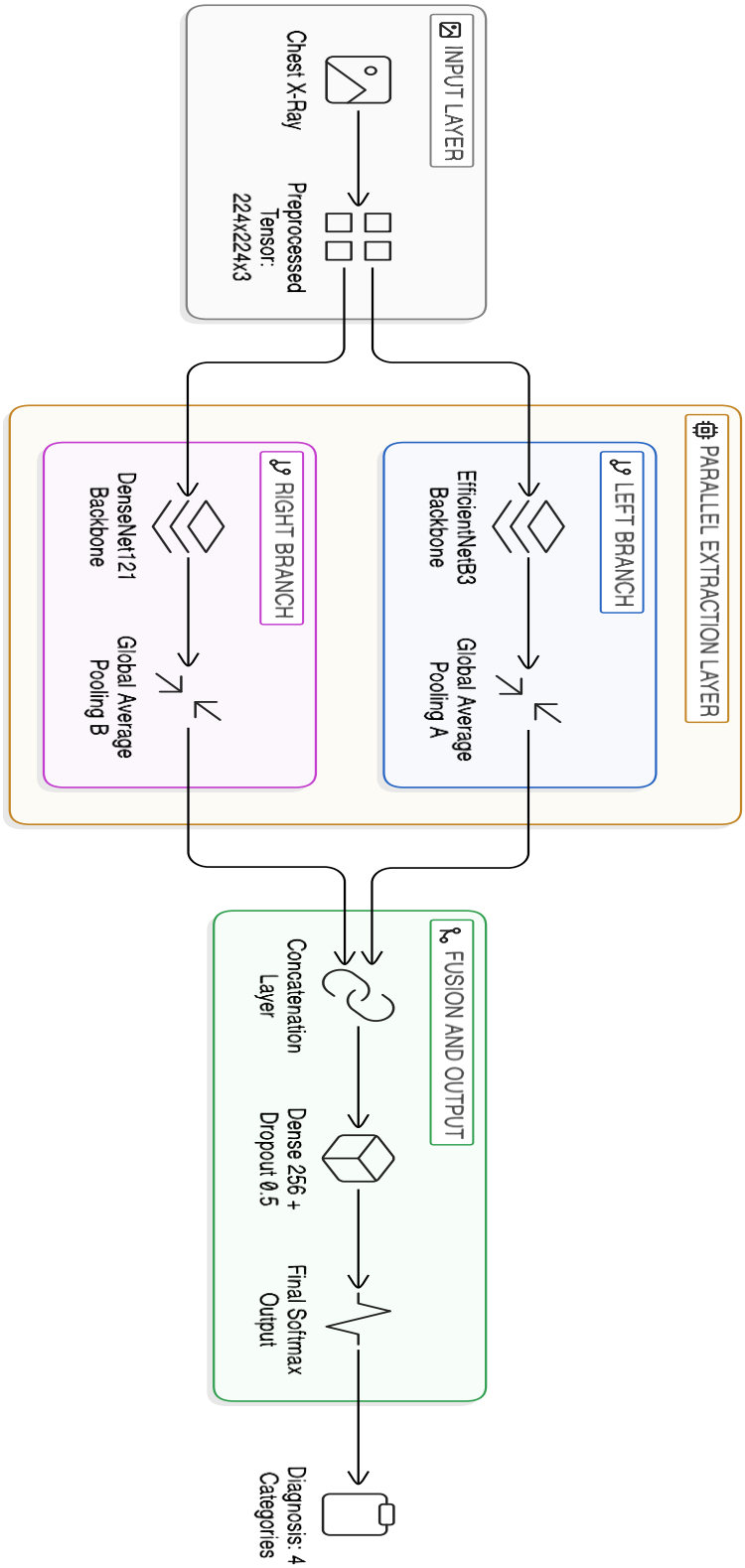
- **Log Processing:** A serverless function monitors the logs in real-time. If it detects a "High Confidence Tuberculosis" prediction, it can trigger an immediate high-priority alert to the medical team's dashboard.

- **Database Triggers:** When a new diagnostic record is written to the database, a serverless trigger automatically updates the global "Disease Trend" statistics used for public health monitoring.

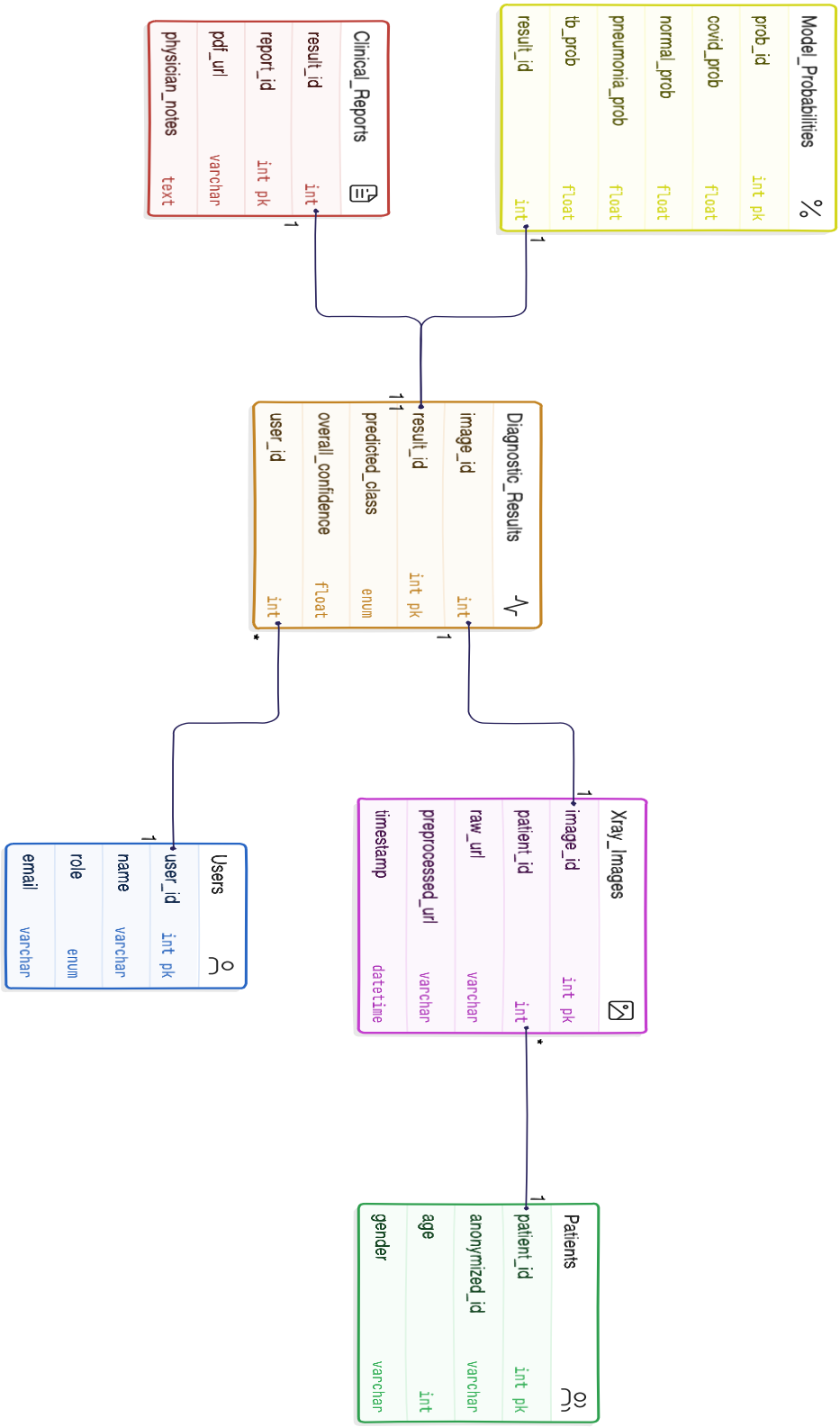
1.3.3 Edge Inference (Future Scope)

The serverless layer is also designed to eventually support "Edge Functions," where lightweight versions of the model (quantized versions) can be pushed closer to the user to reduce latency in regions with poor internet connectivity.

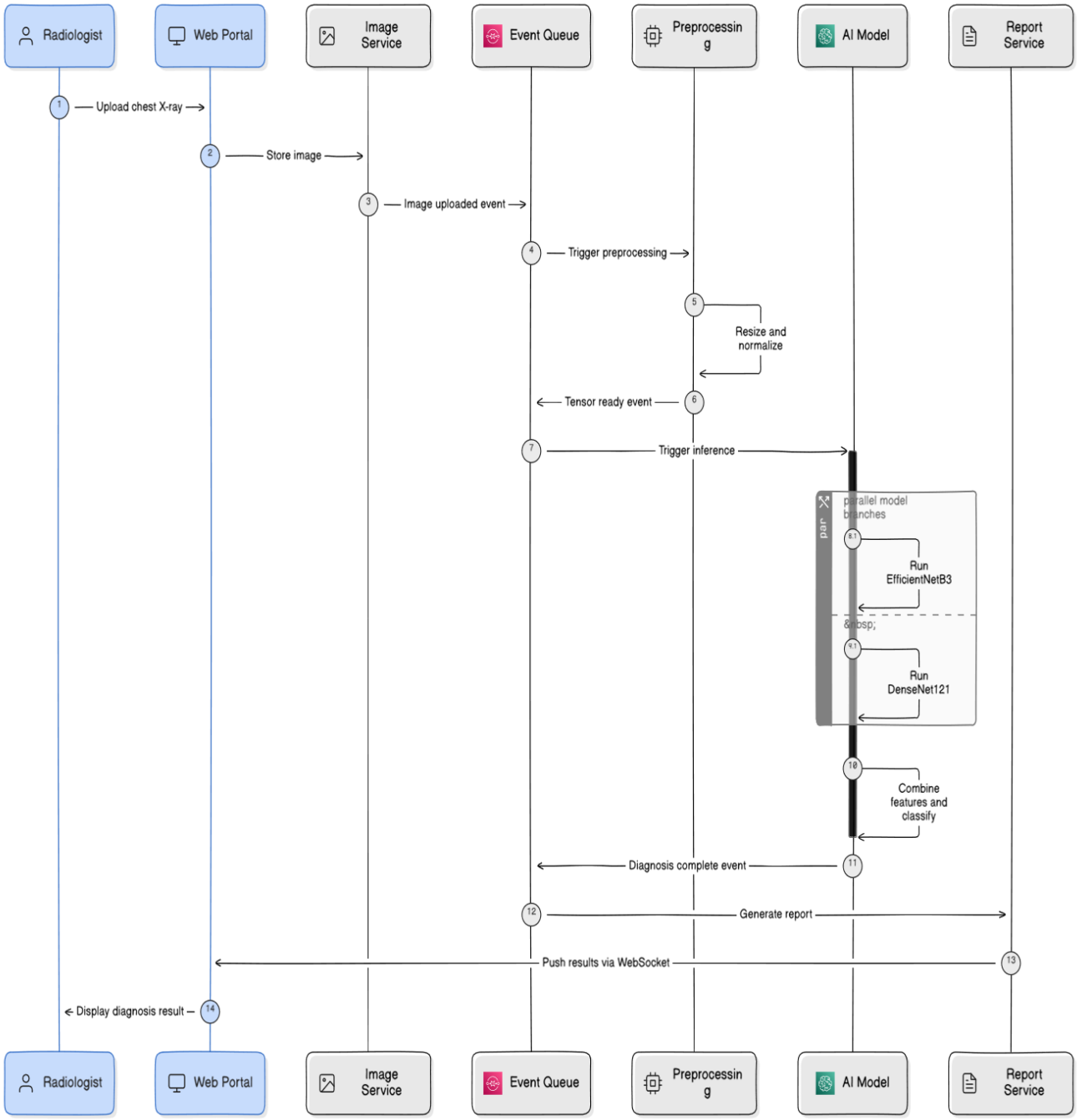
1. Database:



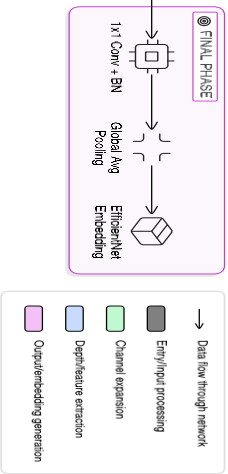
Architecture Diagram


Model_Probabilities
prob_id int pk
covid_prob float
normal_prob float
pneumonia_prob float
tb_prob float
result_id int
Clinical_Reports
result_id int
report_id int pk
pdf_url varchar
physician_notes text
Diagnostic_Results
image_id int
result_id int pk
predicted_class enum
overall_confidence float
user_id int
Xray_Images
image_id int pk
patient_id int
raw_url varchar
preprocessed_url varchar
timestamp datetime
Users
user_id int pk
name varchar
role enum
email varchar
Patients
patient_id int pk
anonymized_id varchar
age int
gender varchar

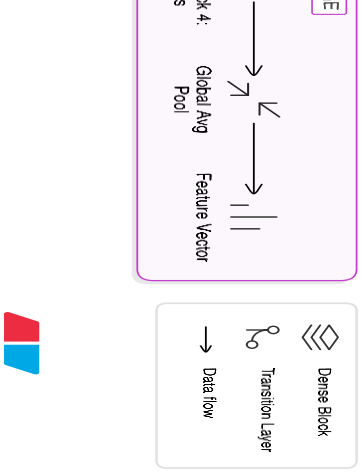
ER DIAGRAM



Sequence Diagram



EfficientNetB3 Architecture



DenseNet121Architecture

3. Data Exchange Contract

The Data Exchange Contract establishes the formal protocols and operational parameters for data movement across the TB and Lung Disease Diagnostic System. In a high-stakes clinical environment, this contract ensures that every interaction—from the initial X-ray upload to the complex fusion of neural embeddings—is performed with maximum integrity, minimal latency, and full auditability.

3.1 Frequency of Data Exchanges

The frequency of data exchange is categorized by the specific operational state of the system, balancing real-time responsiveness for doctors with the high-throughput requirements of batch analysis.

- **Real-Time Transactional Exchanges (High Frequency):**
    - **Context:** Individual patient diagnosis via the Web Dashboard.
    - **Frequency:** Sub-second latency. Every time a user initiates a diagnostic request, an immediate chain of exchanges occurs: Image Upload $\rightarrow$ Preprocessing $\rightarrow$ Inference.
    - **Throughput:** Designed to support a peak clinical load of 60-100 diagnostic runs per hour per medical unit without degradation in response time.
- **Asynchronous Processing Cycles (Medium Frequency):**
    - **Context:** Batch validation of clinical archives or processing the 1,984-image test set.
    - **Frequency:** Continuous streaming. During batch mode, data is exchanged at the maximum capacity of the GPU-optimized **Inference Service**. The system maintains a constant flow of tensors from the storage layer to the hybrid model to ensure 100% compute utilization.

- **System Synchronization & Audit Logs (Low Frequency):**
    - **Context:** Database reconciliation and performance monitoring.
    - **Frequency:** Daily (24-hour cycles). Every night, the system performs a low-priority exchange of diagnostic logs and model confidence trends. This data is synced to the central database to update the long-term performance metrics (e.g., verifying the project's consistent ~95.5% test accuracy over time).

3.2 Data Sets (Logical Schemas)

To ensure the **EfficientNetB3 and DenseNet121** backbones receive consistent inputs, the system enforces four primary data set schemas. Any deviation from these schemas triggers an immediate validation error.

1. **Clinical Ingestion Set (The "Raw" Data):**
    - **Type:** Unstructured Binary + Metadata.
    - **Fields:** patient_id (Encrypted UUID), raw_image_blob (Source file), study_date (ISO-8601), and equipment_type (e.g., CR, DX).
    - **Contract:** The Image Service must accept various formats (JPG, PNG) and verify they are not corrupted before moving them to the next phase.
2. **Standardized Inference Tensor (The "Deep Learning" Data):**
    - **Type:** Structured Multi-dimensional Array.
    - **Shape:** (1, 224, 224, 3).

- **Normalization:** Floating-point precision with a range of [0.0, 1.0].

- **Contract:** This set is the "contractual" input for the **Hybrid Model**. The Preprocessing Service is solely responsible for resizing and applying the $1/255$ scaling logic to convert raw images into this standardized tensor format.

3. **Softmax Probability Payload (The "Insight" Data):**

    - **Type:** Float Array (Vector).

    - **Indices:**

        - Index 0: COVID-19 Probability

        - Index 1: Normal Probability

        - Index 2: Pneumonia Probability

        - Index 3: Tuberculosis (TB) Probability

    - **Contract:** The sum of these four float values must always equal 1.0. This ensures the diagnostic logic remains a closed-set classification as per the project requirements.

4. **Reporting Aggregate (The "Output" Data):**

    - **Type:** Structured JSON / PDF Document.

    - **Fields:** diagnostic_label (String), confidence_score (Percentage), timestamp, and model_version.

3.3 Mode of Exchanges

The architecture uses a multi-modal approach to exchange data, choosing the most efficient communication path based on the size and urgency of the data.

- **RESTful API Interface (Synchronous Exchange):**
    - **Protocol:** HTTPS / JSON.
    - **Usage:** Used for the "Control Plane." When the Radiologist hits "Diagnose," a REST call is used to signal the intention to the system. This provides an immediate acknowledgement that the request has been received and queued.
- **Distributed Message Queue (Asynchronous Exchange):**
    - **Protocol:** AMQP (e.g., RabbitMQ) or Kafka.
    - **Usage:** Used for the "Data Plane." Because the hybrid model requires high compute time to run the parallel EfficientNet and DenseNet branches, the **Inference Tensor** is passed through a queue. This decouples the web server from the GPU server, preventing "Timeout" errors during peak hospital hours.
- **Blob Storage Reference (Pointer-Based Exchange):**
    - **Protocol:** S3-Compatible / Signed URLs.
    - **Usage:** Used for the heavy image files. Instead of moving 5MB images between services (which consumes memory), services exchange a "Signed URL" (a pointer). The AI Engine uses this pointer to pull the image directly into GPU memory, significantly reducing the overhead on the internal network.
- **WebSockets (Real-time Feedback):**

- **Protocol:** WSS (Secure WebSockets).

- **Usage:** Since the inference is asynchronous, the system uses WebSockets to "push" the final diagnosis back to the doctor's screen the millisecond the model finishes calculation. This creates a "Live" feel for the user without requiring them to refresh their browser.